

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title: Cloud Image Registry and Distribution

Date: 2026/6/6

Class: 1

Name: Xue Zhengyang

ID: 202383910028

一、Experimental purpose

Practice Version Control: Learn to use Docker Tags to manage different iterations of a cloud application.

Validate Decoupled Deployment: Simulate a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.

Security Best Practices: Implement Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

二、Experiment Contents

In this lab, you will transition from a "Standalone Developer" to a "Cloud

Engineer." You will:

- Configure secure authentication with GitHub Container Registry (GHCR).
- Tag and push your web application image to the cloud.
- Perform a version upgrade (v1.0 to v2.0) and observe the Layered Storage mechanism.
- Perform a "Clean - room Deployment" by pulling your image onto a fresh environment after wiping local data.

三、 Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Secure Authentication (PAT Generation)

Cloud platforms use Tokens for automation and security.

1.1 Generate Token

Navigate to GitHub Settings -> Developer settings -> Personal access tokens -> Tokens (classic).

1.2 Permissions

Click Generate new token (classic). Select the scope write:packages (this

automatically includes repo and read:packages).

1.3 Save Token

Copy the generated token immediately. You will not be able to see it again.

1.4 Login via Terminal

In your Codespaces terminal, run and **explain** the following code:

```
export CR_PAT=YOUR_COPIED_TOKEN  
echo $CR_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME  
--password-stdin
```

Observation: You should see Login Succeeded.

Note: You may see a warning about "credentials stored unencrypted."

This is normal in a Cloud IDE environment like Codespaces. As long as you see "Login Succeeded," you are ready for Step 2.

Step 2: Tagging for the Cloud

Docker images require a specific naming convention to be recognized by cloud registries.

2.1 Build the Docker Image cloud-lab-image:v1 based on lab 2.

2.2 Apply Tag: Convert your local image into a cloud-ready format.

Note: Your GitHub username must be in LOWERCASE

```
docker tag cloud-lab-image:v1
```

```
ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

Step 3: Pushing the Image (The "Ship" Phase)

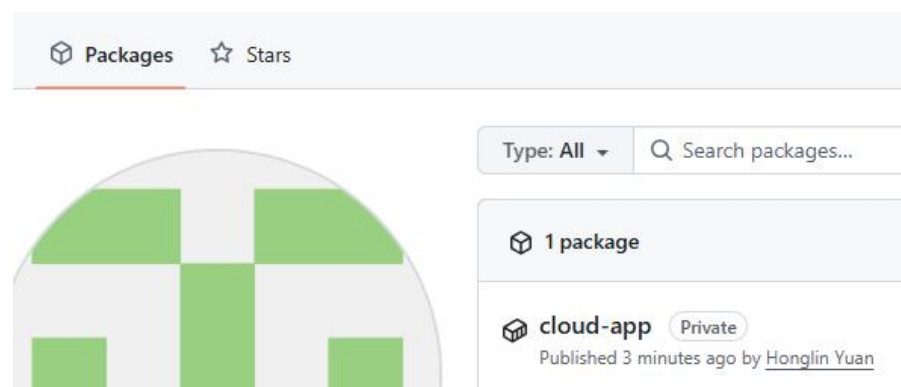
Upload your packaged application to GitHub's global infrastructure.

3.1 Push:

```
docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

3.2 Verify:

Go to your GitHub Profile -> Packages. You should see cloud - app listed.



Step 4: Iteration & Layer Analysis (v2.0)

Cloud computing relies on incremental updates.

4.1 Modify Code

Open index.html and change a visible element (e.g., change the background color or update text to "Welcome to Version 2.0").

4.2 Rebuild & Retag

```
docker build -t cloud-lab-image:v2 .
```

```
docker tag cloud-lab-image:v2
```

ghcr.io/YOUR_USERNAME/cloud-app:v2.0

4.3 Push v2.0: Run docker push

ghcr.io/YOUR_USERNAME/cloud-app:v2.0

Observation: Notice that some lines say Layer already exists. This is Docker's efficiency—only the changed "layer" (the HTML) is uploaded, saving time and bandwidth.

Step 5: Disaster Recovery (The "Pull" Test)

Prove that your application is now independent of your local machine.

5.1 Wipe Local Data: Delete all local images and containers to simulate a new server.

Warning: This removes all local images

docker system prune -a

5.2 Remote Deployment: Deploy directly from the cloud without any source code files.

docker run -d -p 8888:80

ghcr.io/YOUR_USERNAME/cloud-app:v1.0

5.3 Verify: Access the service via the "Ports" tab on port 8888.

5.4 Run: docker run -d -p 8888:80 ghcr.io/honglinyuan/cloud-app:v2.0

You will see Bind for 0.0.0.0:8888 failed: port is already allocated. Find methods to solve this problem.

Step 6: Image Optimization - "Standard vs. Alpine" (15 mins)

In cloud computing, image size directly impacts storage costs and

deployment speed (Cold Start time).

6.1 Compare Base Images

Create two different Dockerfiles to see the impact of OS selection:

6.1 Create a file named Dockerfile.alpine with the following content:

```
FROM nginx:alpine
```

```
COPY . /usr/share/nginx/html
```

6.2 Build both versions:

```
docker build -t cloud-app:standard -f Dockerfile .
```

```
docker build -t cloud-app:alpine -f Dockerfile.alpine .
```

6.3 Size Comparison

```
Run : docker pull nginx:latest
```

```
docker pull nginx:alpine
```

```
docker images | grep nginx
```

compare the size of cloud - app:standard vs. cloud - app:alpine.

Report Task: Record the exact sizes in your lab report. Explain why

"Alpine" is the industry standard for production microservices.

Step 7: Use git to save your changes.

- `git add .` (Gather all your lab changes)
- `git commit -m "Finished Lab 03 "` (Seal the package)
- `git push` (Ship it to GitHub for permanent storage)

四、Reflection

1. Problems Encountered and Proposed Solutions
2. Efficiency: Why did Step 4.3 (Pushing v2.0) upload much faster than Step 3.1? Relate this to the concept of Layer Caching.
3. Security: Why is using a Personal Access Token (PAT) more secure than using your actual GitHub password in a terminal?

Laboratory Report of Nanjing University of Information Science and Technology

Experiment Title: Cloud Image Registry and Distribution

Date: 2026/6/6

Class: 1

Name: Xue Zhengyang

ID: 202383910028

1. Experimental Purpose

The purpose of this experiment was to practice version control using Docker tags to manage different iterations of a cloud application, validate decoupled deployment by simulating a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code, and implement security best practices using Personal Access Tokens instead of plaintext passwords for authentication with GitHub Container Registry.

2. Detailed Steps Summary

Step 1: Secure Authentication. I navigated to GitHub Settings, generated a classic Personal Access Token with write packages scope, and saved it securely. In

the PowerShell terminal, I exported the token as an environment variable and executed `docker login ghcr.io` with my GitHub username in lowercase. The terminal returned Login Succeeded, confirming successful authentication to GitHub Container Registry.

Step 2: Tagging for the Cloud. I verified that the local image `cloud-lab-image:v1` existed from the previous lab. I then applied the cloud registry naming convention by tagging it as `ghcr.io/dlyfyh/cloud-app:v1.0`, ensuring the username was entirely in lowercase.

Step 3: Pushing the Image. I executed `docker push` to upload the packaged application to GitHub's global infrastructure. After the upload completed, I verified the package by navigating to my GitHub Profile and checking the Packages tab, where `cloud-app` was successfully listed.

Step 4: Iteration and Layer Analysis. I modified the `index.html` file to change the background color and update the text to Welcome to Version 2.0. I rebuilt the image as `cloud-lab-image:v2`, retagged it as `ghcr.io/dlyfyh/cloud-app:v2.0`, and pushed it to the registry. During the push, the terminal displayed Layer already exists for multiple layers, demonstrating that Docker only uploaded the changed layer while reusing the existing ones from `v1.0`.

Step 5: Disaster Recovery. I simulated a new server environment by executing `docker system prune -a` to remove all local images and containers. I then deployed directly from the cloud using `docker run` with the `v1.0` image on port 8888. The service was accessible via localhost. To test `v2.0`, I encountered a port conflict because `v1.0` was still running on port 8888. I stopped the `v1.0` container and then deployed `v2.0`, successfully verifying both versions from the cloud without any local source code.

Step 6: Image Optimization. I created a new `Dockerfile.alpine` using `nginx:alpine` as the base image. I built both the standard version and the Alpine version, then

compared their sizes using docker images. The Alpine-based image was significantly smaller than the standard version. Alpine Linux is the industry standard for production microservices because it has a minimal footprint, reduces cold start time, lowers storage costs, and presents a smaller attack surface due to fewer included components.

Step 7: Git Version Control. I gathered all lab changes using `git add`, committed them with the message `Finished Lab 03`, and pushed the commit to the remote GitHub repository for permanent storage.

3. Problems Encountered and Proposed Solutions

Problem 1: Directory path errors due to spaces. When navigating to the project directory in PowerShell, my username contains a space, which caused the `cd` command to fail because the path was interpreted as multiple arguments. I solved this by wrapping the entire path in double quotes.

Problem 2: Missing arguments in `docker tag`. I initially ran `docker tag cloud-lab-image:v2` without providing the target image name, which resulted in an error stating that two arguments are required. I corrected this by adding the full target registry path `ghcr.io/dlyfyh/cloud-app:v2.0`.

Problem 3: Container exited immediately after running. After pulling the `v1.0` image from the cloud during the disaster recovery step, the container exited immediately with status `Exited (1)`. The docker logs revealed an Nginx configuration error: `host not found in upstream web1:5000 in default.conf`. This happened because the original local project directory contained an incorrect Nginx configuration file that was copied into the image during the initial build. I solved this by removing the erroneous configuration files from the project directory, rebuilding the image with a clean Dockerfile containing only the necessary instructions, and repushing the corrected image to overwrite the faulty version in the registry.

Problem 4: Browser refused connection to localhost. Initially, I could not access the running container via localhost:8888 because the container had already crashed due to the Nginx configuration error mentioned above. Once the image was rebuilt correctly, the container remained running and the browser successfully displayed the webpage.

Problem 5: Placeholder username not replaced. When running the v2.0 container, I used `YOUR_GITHUB_USERNAME` instead of my actual username, causing Docker to reject the reference format because it requires lowercase. I fixed this by replacing the placeholder with my actual GitHub username `dlyfyh`.

Problem 6: Dockerfile.alpine encoding issues. When creating the Alpine Dockerfile, I accidentally pasted the Dockerfile content directly into the PowerShell terminal instead of writing it to a file, which caused a parser error because `FROM` is not a PowerShell command. I resolved this by using PowerShell's `Out-File` command to properly create the file with correct UTF-8 encoding without BOM.

Problem 7: Git push authentication. During the final git push step, Git prompted me to complete authentication in the browser. I followed the browser prompt to authorize the push, after which the commit was successfully uploaded to the remote repository.

4. Reflection

Question 1: Why did Step 4.3 pushing v2.0 upload much faster than Step 3.1?

The v2.0 push was significantly faster because of Docker's layered storage mechanism and layer caching. A Docker image consists of multiple read-only layers, each corresponding to a specific instruction in the Dockerfile. When I built v2.0, only the layer containing the modified `index.html` file changed. All other layers, including the base `nginx:alpine` image and any unchanged dependencies, remained identical to v1.0. During the push to GitHub Container Registry, Docker detected that these

unchanged layers already existed in the remote repository and therefore did not re-upload them. The terminal output explicitly showed Layer already exists for the majority of layers. Only the new or modified layer needed to be transferred over the network. This incremental update approach saves considerable time and bandwidth, which is a fundamental efficiency principle in cloud computing and container registry management.

Question 2: Why is using a Personal Access Token more secure than using your actual GitHub password in a terminal?

Using a Personal Access Token is more secure for several reasons. First, it follows the principle of least privilege. A PAT can be scoped to specific permissions, such as write:packages only, whereas a GitHub password grants full account access. Second, PATs are independently revocable. If a token is compromised or no longer needed, it can be deleted without changing the main account password or affecting other applications. Third, PATs support expiration dates, allowing administrators to enforce regular rotation and reduce the risk of long-term credential exposure. Fourth, GitHub maintains an audit trail for PAT usage, making it easier to detect suspicious activity. Finally, using a token prevents the actual account password from being exposed in terminal history, environment variables, or scripts, thereby protecting the primary authentication credentials even if the development environment is compromised.

5. Conclusion

This laboratory exercise successfully transformed my understanding from a standalone developer workflow to a cloud engineer workflow. I configured secure authentication with GitHub Container Registry using Personal Access Tokens, tagged and pushed Docker images to a cloud registry, performed version upgrades while observing the efficiency of Docker's layered storage mechanism, and validated

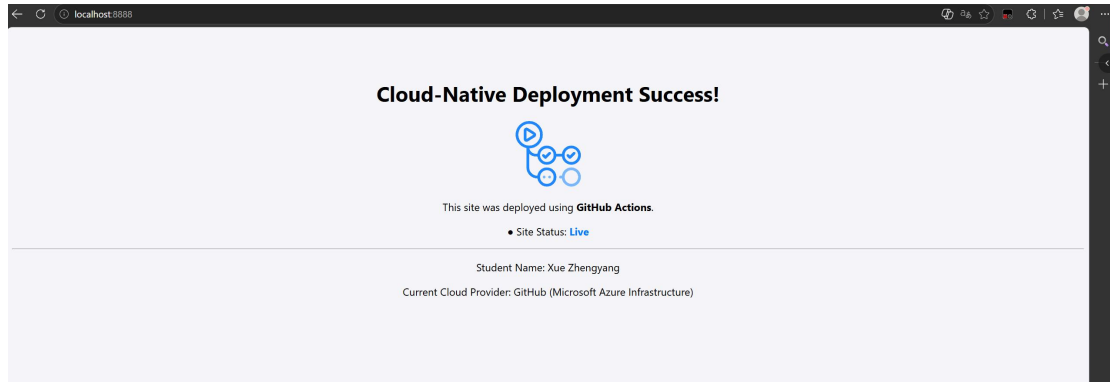
decoupled deployment through a disaster recovery simulation. The experiment also demonstrated the importance of image optimization by comparing standard and Alpine base images, revealing why Alpine is the preferred choice for production microservices. Through troubleshooting various issues including path handling, container configuration errors, and authentication flows, I gained practical experience in diagnosing and resolving real-world cloud deployment problems. Overall, this lab reinforced the core cloud computing concepts of immutability, scalability, security, and efficiency.

Screenshot:

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
2d07493dd1b1   ghcr.io/dlyfyh/cloud-app:v1.0      "/docker-entrypoint..." 22 minutes ago Up 22 minutes 0.0.0.0:8888->80/tcp, [::]:8888->80/tcp
cba947608522   docker.gitea.com/gitea:1.26.1      "/usr/bin/entrypoint..." 5 weeks ago    Up 2 hours    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker stop CONTAINER_ID
Error response from daemon: No such container: CONTAINER_ID
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> |
```

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker run -d -p 888:80 ghcr.io/dlyfyh/cloud-app:v2.0
Unable to find image 'ghcr.io/dlyfyh/cloud-app:v2.0' locally
v2.0: Pulling from dlyfyh/cloud-app
c1cc92ce890d: Pull complete
8117d74e2950: Download complete
Digest: sha256:60098e0d57dc6b6e9b9a1fb0deaf0736806741014a100f97f6e15ed1f81551db
Status: Downloaded newer image for ghcr.io/dlyfyh/cloud-app:v2.0
9bdcd84c5d7fb0a0b460d35bad695caffd9847af4bbb983ff60aa78f3bf5a923
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint loving_ellis (8415fb9db34ea1088eafe2b67b9c0a4309f0eaf9fcbea8c51f94b567293d6f9f): Bind for 0.0.0.0:8888 failed: port is already allocated

Run 'docker run --help' for more information
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>
```



```
> docker run -d -p 88
88:80 ghcr.io/dlyfyh/cloud-app:v1.0
Unable to find image 'ghcr.io/dlyfyh/cloud-app:v1.0' locally
v1.0: Pulling from dlyfyh/cloud-app
8d0ed7badb72: Pull complete
ff048f1f2159: Pull complete
ce776bbcdad0d: Pull complete
4677c2a9a3d4: Pull complete
677c63196868: Pull complete
3531af2bc2a9: Pull complete
801a1ad15b4e: Pull complete
dc4867a1ab4c: Pull complete
85c66128325a: Pull complete
1856d06080f4: Pull complete
bde3ea3dae5d: Download complete
Digest: sha256:f4293d092be80f1b8246a7e8a731a800258461e3c376d00ce326ab1e58e7eb80
Status: Downloaded newer image for ghcr.io/dlyfyh/cloud-app:v1.0
ae7eaaea437ba0adf0b9ed56b38bec31ea22c4b00ab9097d3da618d3bfbe01c5
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>
```

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker system prune -a
WARNING! This will remove:
 - all stopped containers
 - all networks not used by at least one container
 - all images without at least one container associated to them
 - all build cache
```

Are you sure you want to continue? [y/N] y

Deleted Containers:

87a8c09348148c274b32f3add6e813aecebe379cd92d5dd64df8a4d3a53f3aa9
30a3878f0a76ad3f8174822fad4b9a73471a7ec9b3438d46bec065e2839331d4

Deleted Images:

untagged: cloud-lab-image:v1

deleted: sha256:1135ac155af6bb52fe2372fe0c427f358241d55097e2a7dd862fa1eee845557d

deleted: sha256:6c6fdb580c132c6748fb748f11bd7f0ad11c471886b25b100d94cd2c6cca3205

deleted: sha256:8e11574339192799ac8d73023f4c0d3e12cc22d89843fccd3aa9074fac7161cc

untagged: ghcr.io/dlyfyh/cloud-app:v1.0

deleted: sha256:f4293d092be80f1b8246a7e8a731a800258461e3c376d00ce326ab1e58e7eb80

deleted: sha256:fd224b27794d23941581e0e07aea131b4a7234604cb940bee952b707f099bbf1

deleted: sha256:7ea724e4bf2c4ccb3703d541510e5e52278e47d4faa0690502772babdf796ef

untagged: ghcr.io/dlyfyh/cloud-app:v2.0

untagged: envoyproxy/envoy:v1.36.4

deleted: sha256:08acc53db6f4ac46e77e36b203c5254b4a2549869314bb54cf97aab81baa4566

deleted: sha256:ae31562b8cede20913a2d3d6a4f44c8479a50551e033cb8ef7bb8e38cec4b573

untagged: cloud-lab-image:v2

deleted: sha256:60098e0d57dc6b6e9b9a1fb0deaf0736806741014a100f97f6e15ed1f81551db

deleted: sha256:fee44a64a3de22a89de9898329687053c70a1452bba85d1515bbf90ccd31b287

deleted: sha256:73e108531c868e804c34cbc845fa2d10a66de638257ff9d545d4fd5d31bb91f0

Deleted build cache objects:

t6g0rc400r252ofssu912pr74

pgab3c3audqtcutisrrac74jd

wt5ekdt2zoqlkdaw8a44yty51

rhs5z2gvmvrlxchsv7fooxn7x8

8yvw1uuo70ntxuzzd5n7nszmp

ycovk1n7263kscyemre49qosx

tviyxefg8upbn13xlky0y2f8i

adkt17y55pcun4dkgw808ltq2

l24nnl7bh8v5gfusc0xajcilj

mp26nfp9ihatbs1gj1gkbf9pi

t675f7ug6s575vq7r0qfjdhj9

v2bujakanozoad7uk3608o0bs

7niqye777hivdubeasxzu8pxz

hw891s8vzzi4uoo8sf76a6agc

kzp9my7q2a9dxz7icjiq2ddyn

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker tag cloud-lab-image:v2 ghcr.io/dlyfyh/cloud-app:v2.0
```

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker push ghcr.io/dlyfyh/cloud-app:v2.0
```

The push refers to repository [ghcr.io/dlyfyh/cloud-app]

c1cc92ce890d: Pushed

de1b677d8c00: Pushed

94d083cf706a: Pushed

a5008f4a4b25: Pushed

abaae85d1626: Pushed

6a0ac1617861: Pushed

43f834d60d8a: Pushed

8117d74e2950: Pushed

e654dbbbb9e1: Pushed

fbaed3f7fcbe: Pushing 20.28MB/20.28MB

```

PS C:\WINDOWS\system32> cd "C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github"
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker build -t cloud-lab-image:v2 .
[+] Building 18.5s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 319B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  6.9s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 1.06kB                                0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621 10.1s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e 0.1s
=> sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 4058 / 4058 0.8s
=> sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 6288 / 6288 1.4s
=> sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 19.92MB / 20.28MB 10.8s
=> sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 1.21kB / 1.21kB 2.6s
=> sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 957B / 957B 1.5s
=> sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 1.40kB / 1.40kB 1.3s
=> sha256:abaae85d162e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 1.88MB / 1.88MB 4.0s
=> extracting sha256:abaae85d162e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 0.2s
=> extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 0.0s
=> extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 0.0s
=> extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 0.0s
=> extracting sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 0.1s
=> extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 0.0s
=> extracting sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 0.4s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html      0.3s
=> exporting to image                                           0.6s
=> => exporting layers                                           0.2s
=> exporting manifest sha256:fee44a643de22a89de9898329687053c70a1452bba85d1515bbf90ccd31b287 0.0s
=> exporting config sha256:ae368e3ab47b35676f10fc0064f2da21c35b25e43bea750547ddd418ab795d18 0.0s
=> exporting attestation manifest sha256:73e108531c868e804c34c845fa2d10a66de638257ff9d545d4fd5d31bb91f0 0.1s
=> exporting manifest list sha256:60098e0d57dc6b6e9b9a1fb0dea0f736806741014a100f97f6e15ed1f81551db 0.0s
=> naming to docker.io/library/cloud-lab-image:v2             0.0s
=> unpacking to docker.io/library/cloud-lab-image:v2          0.1s
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>

```

```

<> index.html M X
C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> <> index.html <> html <> head <> title
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Welcome to Version 2.0</title>
7 <style>
8     body {
9         font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
10        text-align: center;
11        padding-top: 50px;
12        background-color: #f4f4f9;
13    }
14    .status {
15        color: #007bff;
16        font-weight: bold;
17    }
18 </style>
19 </head>
20 <body>
21 <h1>Cloud-Native Deployment Success!</h1>
22
23 
24
25 <p>This site was deployed using <strong>GitHub Actions</strong>.</p>
26
27 <p>• Site Status: <span class="status">Live</span></p>
28
29 <hr>
30
31 <p>Student Name: Xue Zhengyang</p>
32 <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
33 </body>
34 </html>
35

```


DLYFYH

🔍

👤

+

🔄

🔗

📁

📧

📖 Overview

📦 Repositories 2

📁 Projects

📦 Packages

☆ Stars

📦 cloud-app v2.0

Private

Latest

Installation

OS / Arch 2

Learn more about packages

📄 Install from the command line

```
$ docker pull ghcr.io/dlyfyh/cloud-app:v2.0
```

Recent tagged image versions

v2.0

Published 7 days ago · Digest ...

📄 0

v1.0

Published 7 days ago · Digest ...

📄 0

View and manage all versions

Last published

7 days ago

Total downloads

0

⚙️ Package settings

DLYFYH

🔍

👤

+

🔄

🔗

📁

📧

📖 Overview

📦 Repositories 2

📁 Projects

📦 Packages

☆ Stars

Xue Zhengyang

DLYFYH

Edit profile

Type: All 🔍

Visibility: All

Sort by: Most downloads

📦 1 package

📦 cloud-app

Private

Published last week by Xue Zhengyang

📄 0

```
PS C:\WINDOWS\system32> docker push ghcr.io/dlyfyh/cloud-app:v1.0
The push refers to repository [ghcr.io/dlyfyh/cloud-app]
bde3ea3dae5d: Already exists
8d0ed7badb72: Layer already exists
4677c2a9a3d4: Layer already exists
ff048f1f2159: Layer already exists
677c63196868: Layer already exists
ce776bbcdad0: Layer already exists
dc4867a1ab4c: Layer already exists
85c66128325a: Layer already exists
3531af2bc2a9: Layer already exists
801a1ad15b4e: Layer already exists
1856d06080f4: Layer already exists
v1.0: digest: sha256:f4293d092be80f1b8246a7e8a731a800258461e3c376d00ce326ab1e58e7eb80 size: 856
PS C:\WINDOWS\system32>
```

```
PS C:\WINDOWS\system32> docker images | findstr "cloud-app"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/dlyfyh/cloud-app:v1.0          f4293d092be8          237MB          63MB          U
```

```
PS C:\WINDOWS\system32> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-lab-image:v1	1135ac155af6	238MB	63.1MB	
docker.gitea.com/gitea:1.26.1	d8667667b4cc	245MB	69.3MB	U
docker/desktop-cloud-provider-kind:v0.5.0	4ad59ce20658	595MB	162MB	U
docker/desktop-containerd-registry-mirror:v0.0.3	179a9ea3ae47	50.3MB	16.1MB	U
envoyproxy/envoy:v1.36.4	08acc53db6f4	248MB	62.1MB	U
ghcr.io/dlyfyh/cloud-app:v1.0	f4293d092be8	237MB	63MB	U
kindest/node:v1.34.3	08497ee19eac	1.35GB	395MB	U

```
PS C:\WINDOWS\system32>
```

```
PS C:\WINDOWS\system32> $env:CR_PAT="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"
PS C:\WINDOWS\system32> $env:CR_PAT | docker login ghcr.io -u dlyfyh --password-stdin
Login Succeeded
```

Settings / Developer Settings

Q Type [f] to search

+

Some of the scopes you've selected are included in other scopes. Only the minimum set of necessary scopes has been saved.

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)

Generate new token

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

✓

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

Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

```
Microsoft Windows [版本 10.0.26200.8457]  
(c) Microsoft Corporation。保留所有权利。
```

```
C:\Users\Xue Zhengyang>wmic pagefile list /format:list
```

```
AllocatedBaseSize=14336  
CurrentUsage=852  
Description=D:\pagefile.sys  
InstallDate=20260510085821.990231+480  
Name=D:\pagefile.sys  
PeakUsage=939  
Status=  
TempPageFile=FALSE
```

```
C:\Users\Xue Zhengyang>systeminfo | findstr "虚拟内存"  
虚拟内存：最大值： 30,452 MB  
虚拟内存：可用： 5,576 MB  
虚拟内存：使用中： 24,876 MB
```

```
C:\Users\Xue Zhengyang>s
```

The screenshot shows the GitHub interface for the repository 'cloud-computing-lab-github'. The repository is public and was forked from 'yhldf/cloud-computing-lab-github'. It has 1 branch (main) and 0 tags. The repository is 6 commits ahead of the fork. The commit history shows a recent commit by 'DLVYH' titled 'Finished Lab 03' with a green checkmark, dated 9374382 - 2 minutes ago. The commit message is 'Create static.yml'. The repository also has a README section with a placeholder for a README file. The right sidebar contains sections for 'About' (No description, website, or topics provided), 'Releases' (No releases published), 'Packages' (No packages published), 'Contributors' (No contributors), and 'Languages' (HTML 78.4%, Dockerfile 21.6%).

```

PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> git add .
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> git commit -m "Finished Lab 03"
[main 9374382] Finished Lab 03
 2 files changed, 3 insertions(+), 1 deletion(-)
 create mode 100644 Dockerfile.alpine
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> git push
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 455 bytes | 455.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DLYFYH/cloud-computing-lab-github.git
 13d9b04..9374382 main -> main
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>

```

```

                                docker images | findstr "nginx"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx:alpine                    8b1e78743a03          93.6MB          27MB
nginx:latest                    5aca99593157          241MB           66MB
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker images | findstr "cloud-app"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-app:alpine               f099b80c2ed4          93MB            26.1MB
cloud-app:standard             3c7792516018          92.7MB          26MB
ghcr.io/dlyfyh/cloud-app:v1.0  cad3d3ab7c7f          92.7MB          26MB   U
ghcr.io/dlyfyh/cloud-app:v2.0  60098e0d57dc          92.7MB          26MB   U
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>

```

```

PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker build -t cloud-app:standard -f Dockerfile .
[+] Building 1.3s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 319B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  0.1s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 1.06kB                                0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc586 0.3s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc586 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html      0.2s
=> exporting to image                                             0.4s
=> => exporting layers                                           0.2s
=> => exporting manifest sha256:253af6cbd685bcb97bddcca94e4c1b9fda2e9f6a0866fc 0.0s
=> => exporting config sha256:b5609dea06c904cdf3b25dcca975e31cdfcaabac665312cc 0.0s
=> => exporting attestation manifest sha256:a0843e6d9ff03115ef2bfe5880afc9f3599 0.0s
=> => exporting manifest list sha256:3c77925160182c2be46877eb3305d770361e0b67c7 0.0s
=> => naming to docker.io/library/cloud-app:standard            0.0s
=> => unpacking to docker.io/library/cloud-app:standard          0.0s
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker build -t cloud-app:alpine -f Dockerfile.alpine .
[+] Building 0.8s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile.alpine       0.0s
=> => transferring dockerfile: 91B                               0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 38.82kB                                0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c951 0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc586 0.0s
=> [2/2] COPY . /usr/share/nginx/html                          0.1s
=> exporting to image                                             0.4s
=> => exporting layers                                           0.1s
=> => exporting manifest sha256:28dc8f42f5a841c818647fe187a809cbe803f1f29a7d2a1 0.0s
=> => exporting config sha256:1911253c5b66135e90e9c91e864a96970421a19c7c2cefa74 0.0s
=> => exporting attestation manifest sha256:e07ea4b762dca175efef5009bdf7d0f6a7d 0.1s
=> => exporting manifest list sha256:f099b80c2ed4d58c7058dfff18644fd40138b2e9068 0.0s
>> => naming to docker.io/library/cloud-app:alpine              0.0s
=> => unpacking to docker.io/library/cloud-app:alpine            0.0s
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>

```

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker pull nginx:latest
latest: Pulling from library/nginx
13fd728be9eb: Pull complete
7f8b1a2b17d8: Pull complete
45381ecb0e2f: Pull complete
5431d0092ffd: Pull complete
830625e1ac85: Pull complete
5b4d6ff92fc4: Pull complete
b4a248c845e5: Pull complete
cc5f57206478: Download complete
5e6b66b5e5f1: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker pull nginx:alpine
alpine: Pulling from library/nginx
ab556c9e4280: Download complete
2d8223aa4623: Download complete
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github>
```

```
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> docker run -d -p 8889:80 ghcr.io/dlyfyh/cloud-app:v2.0
e2927d548a280ca269eba9528fed807f2e296d1f96ea4aff0b2a0bc058904fb4
PS C:\Users\Xue Zhengyang\Lab\Documents\cloud-computing-lab-github> |
```

←

🔍

🌐 localhost:8889

Summarize

🔗

🌟

🔄

🔖

👤

Cloud-Native Deployment Success!



This site was deployed using **GitHub Actions**.

• Site Status: [Live](#)

Student Name: Xue Zhengyang

Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)